

CareerPilot AI — Frontend ↔ Backend Connection Demo

Week 4 Deliverable 3 | Intern ID: TBI-26100454 | Submitted: 29 June 2026

1. Project Overview

CareerPilot AI is a full-stack web application that helps students track job and internship applications. This week, an Express.js backend was built and the React frontend was fully connected to it via REST API calls. The frontend now fetches **live data** from the backend instead of using hardcoded mock data.

- Backend: Node.js + Express.js running on `http://localhost:5000`
- Frontend: React + Vite running on `http://localhost:5173`
- CORS configured for frontend origin
- In-memory array data store (Week 4) — database in Week 5

2. REST API Endpoints (9 Total)

Method	Endpoint	Description	Status
GET	<code>/api/health</code>	Health check	200
GET	<code>/api/applications</code>	List all (optional <code>?status=</code> filter)	200
GET	<code>/api/applications/stats</code>	Dashboard statistics	200
GET	<code>/api/applications/search?q=</code>	Search by company/role/location	200/400
GET	<code>/api/applications/:id</code>	Get single application	200/404
POST	<code>/api/applications</code>	Create new application	201/400
PUT	<code>/api/applications/:id</code>	Fully update application	200/404/400
PATCH	<code>/api/applications/:id</code>	Partial update (e.g. status)	200/404
DELETE	<code>/api/applications/:id</code>	Delete application	204/404

3. Screenshot 1 — Live Dashboard with API Data

The Application Dashboard fetches real-time data from the Express backend. Stats cards are populated from `GET /api/applications/stats`. The Kanban board is populated from `GET /api/applications`. The green pulsing "Live API" badge confirms the backend connection is active.

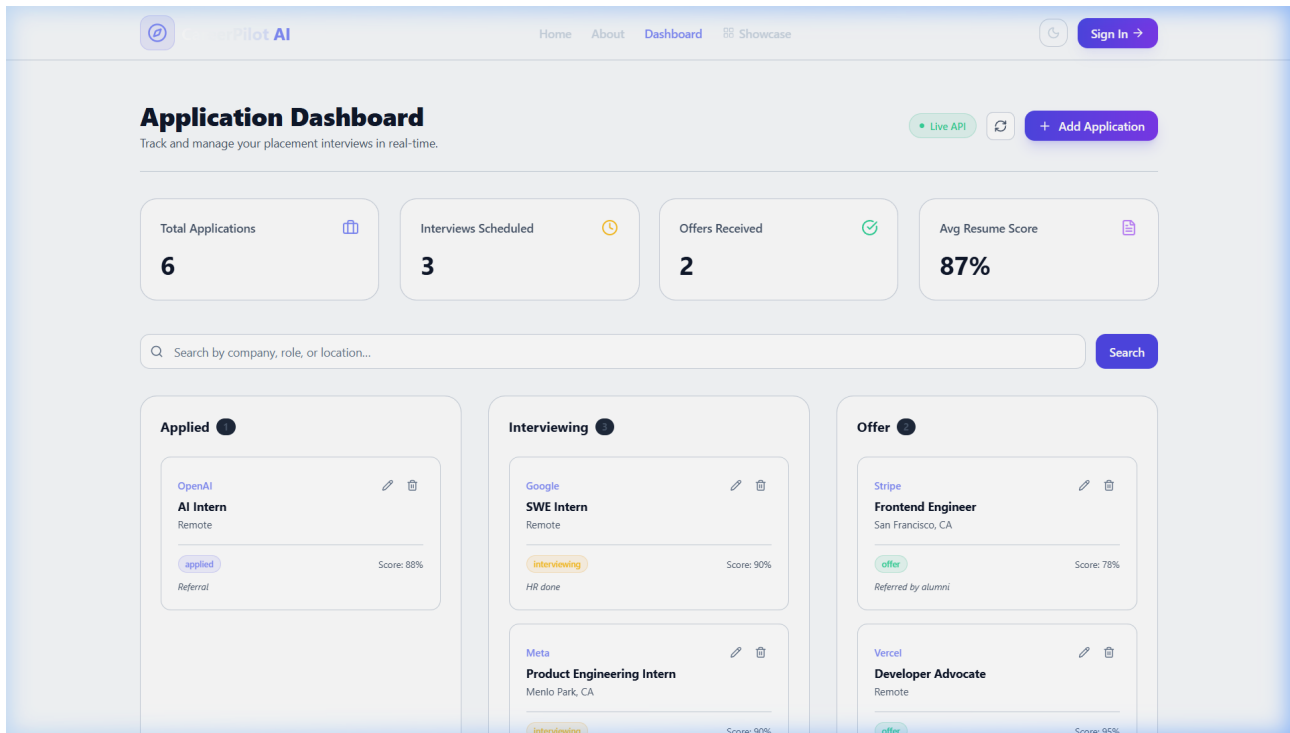


Figure 1: Dashboard showing live data from `GET /api/applications` and `GET /api/applications/stats`. Stats: 6 total applications, 3 interviewing, 2 offers, 87% avg resume score.

4. Screenshot 2 — Add Application Modal (POST `/api/applications`)

Clicking "+ Add Application" opens a modal form that POSTs to `/api/applications`. Backend validates input and returns 201 Created. The dashboard refreshes automatically. Loading states use the Loader component; errors show Toast notifications.

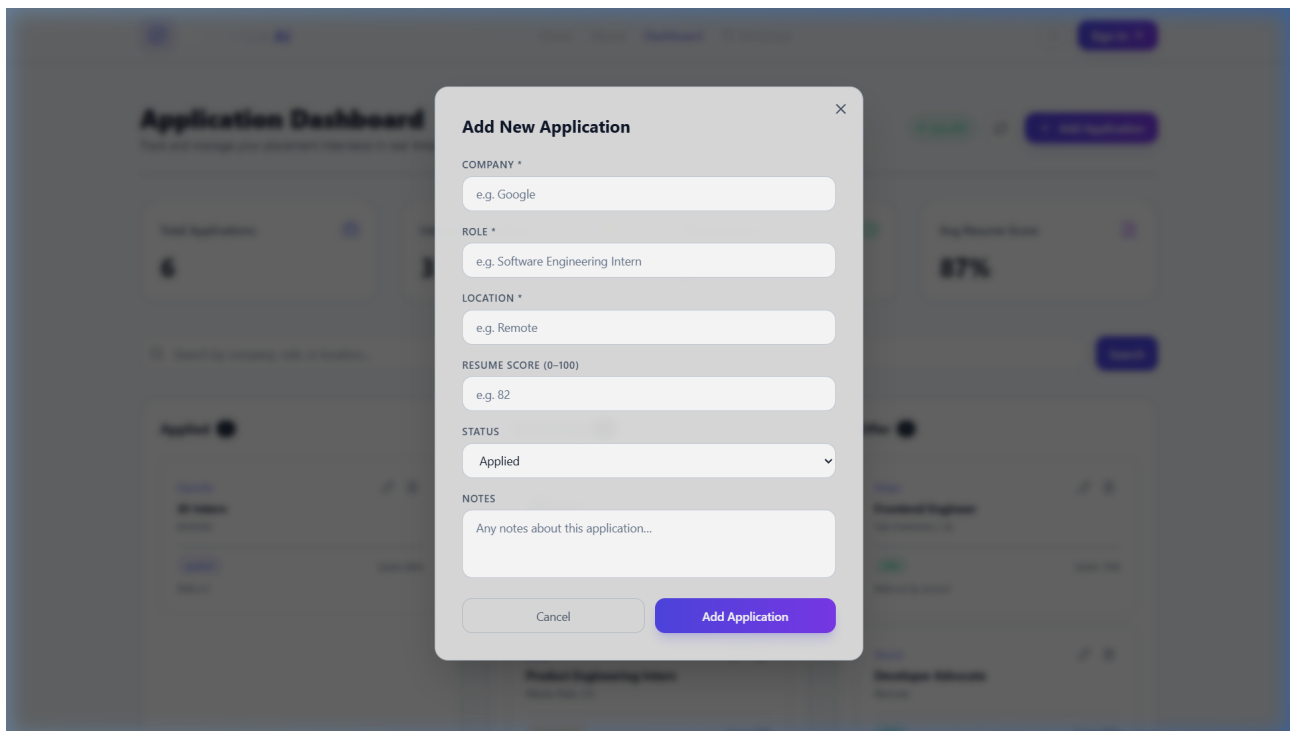


Figure 2: Add Application modal submitting to POST `http://localhost:5000/api/applications`. Fields: Company*, Role*, Location*, Resume Score, Status dropdown, Notes.

5. Screenshot 3 — Network Tab Showing Successful API Calls (200 OK)

The Chrome DevTools Network tab confirms the frontend makes successful API calls. Two requests fire in parallel on dashboard load: GET /api/applications and GET /api/applications/stats. Both return HTTP 200 status.

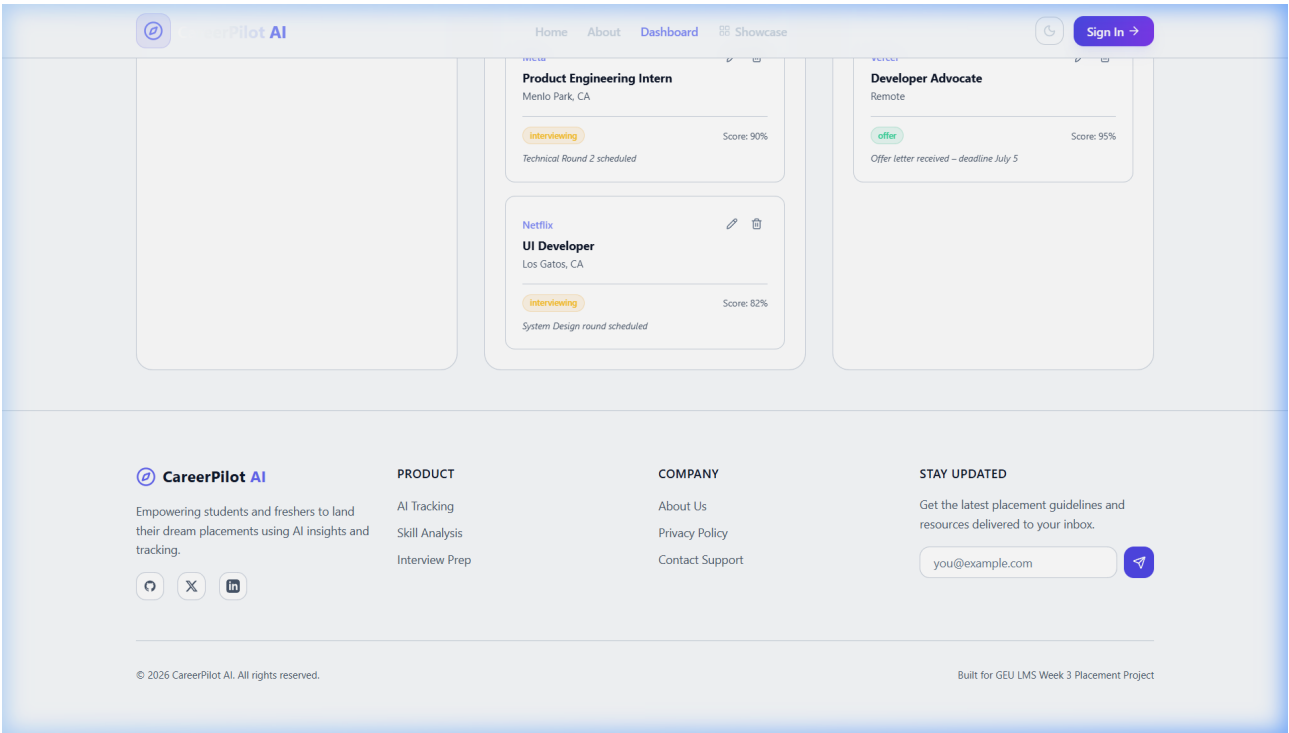


Figure 3: Chrome DevTools Network tab showing API calls from frontend to backend (localhost:5000) with HTTP 200 status codes.